

A Mixed-Methods Study on the Implications of Unsafe Rust for Interoperation, Encapsulation, and Tool Design - Appendix

Contents

1	Recruitment	3
1.1	Email Templates	3
1.1.1	Direct Recruitment	3
1.1.2	Initial Invitation	3
1.1.3	Payment & Forwarding	3
1.1.4	Follow-up Invitation	4
1.1.5	Community Survey Compensation	4
1.2	Forum Posts	4
1.2.1	Interview Study Recruitment Template	5
1.2.2	Community Survey Recruitment Template	5
2	Consent	6
2.1	Screening & Community Surveys	6
2.2	Interviews	8
3	Interviews	9
3.1	Screening Survey	9
3.1.1	Eligibility	9
3.1.2	Background	9
3.1.3	Personal Information	9
3.2	Interview Protocol	10
4	Community Survey	11
4.1	Affirming Consent	11
4.2	Eligibility	11
4.3	Background	11
4.4	Domain	11
4.5	Unsafe Features	12
4.6	UnsafeCell	13
4.7	Calling Unsafe Functions	13
4.8	Intrinsics	14
4.9	ManuallyDrop + MaybeUninit	14
4.10	Box	15
4.11	Why unsafe?	15
4.12	Why unsafe? - No clear alternative	15
4.13	Why unsafe? - Ergonomics	16
4.14	Why unsafe? - Performance	16
4.15	Performance VS Safety	17
4.16	Local Minimal Unsafe	18
4.17	Encapsulation	19
4.18	Encapsulation - Unsafe API	19

4.19	Encapsulation - Unsafe API - Invariants	19
4.20	Encapsulation - Safe API	20
4.21	FFI - Background	20
4.22	FFI - Memory Model	21
4.23	FFI - Bindings - Automation	22
4.24	Validation	23
4.25	Validation - Miri	25
4.26	Community and Culture	25
4.27	Demographics	26
4.28	Personal Information - Consent	27
4.29	Personal Information - Connection	27
4.30	Personal Information - Drawing	28
4.31	Personal Information - Email	28
5	Interrater Reliability	29

1 Recruitment

1.1 Email Templates

1.1.1 Direct Recruitment

The following email was sent to recruit our acquaintances and start the process of snowball sampling.

Hello [*candidate name*],

I'm [*interviewer name*], a PhD student at Carnegie Mellon University's Software and Societal System's Department.

We're studying unsafe Rust, and we think you have relevant experience that we could learn from.

If you're interested, we'd like you to complete a brief screening survey [*link to survey inserted in-line*].

If your responses indicate that you are eligible, then you'll be invited to complete a semi-structured, virtual interview. Interview participants will be compensated \$30 per hour of interview time.

If you're unable to participate, but you know someone we should speak with, forward them a link to our survey!

1.1.2 Initial Invitation

The following email was sent to eligible candidates that had completed the screening survey to invite them to complete an interview.

Hello [*candidate name*],

I'm [*interviewer name*], a PhD student at Carnegie Mellon University's Software and Societal System's Department.

We're studying unsafe Rust, and we think you have relevant experience that we could learn from.

Thank you for taking our screening survey! We'd like to invite you to complete a 1-2 hour, semi-structured interview so that we can learn more about your experiences with unsafe Rust.

You will be compensated \$30 for participation, with an additional \$30 for any additional hour of interview time.

If you're able to participate, select up to two of the time slots in the schedule linked below based on the time you're willing to commit. Make sure that you have a private space available for the interview.

[*insert link to calendar with time slots*]

Looking forward to hearing from you!

[*interviewer signature*]

1.1.3 Payment & Forwarding

Thanking interview participants for their time and providing their compensation, along with instructions for forwarding the information about the study to other participants.

Hello [*candidate name*],

Thank you for participating in our study! Attached you will find your \$30 compensation in the form requested ([*Amazon, Target, Starbucks gift card*]).

If you know of anyone else who we would benefit from interviewing, please send them the following link to our screening survey:

[*link to survey*]

Thanks again!

[*interviewer signature*]

1.1.4 Follow-up Invitation

Prompting participants who have completed one interview to schedule another.

Hello [*candidate name*],

You indicated during our initial interview that you were open to completing a follow-up interview. If you are still open to it, it would be great to hear from you again!

You can select another time slot with the following link:

[*insert link to calendar with time slots*]

Looking forward to hearing from you!

[*interviewer signature*]

1.1.5 Community Survey Compensation

Sent to community survey participants that were randomly chosen to receive compensation.

Hello [*candidate name*],

Thanks for completing our community survey! You were one of the four participants randomly chosen to receive a \$250 gift card. Here is the code for a \$250 gift card to [*Amazon, Target or Starbucks*] as requested in your survey response.

Thanks again!

[*interviewer signature*]

1.2 Forum Posts

Interview Study Recruitment Links:

- [Twitter/X](#)
- [Reddit](#)
- [Rust Forums](#)

Community Survey Recruitment Links:

- [Twitter/X](#)

- [Reddit](#)
- [Rust Forums](#)
- [‘This Week in Rust’](#)

1.2.1 Interview Study Recruitment Template

Title: *Calling Unsafe Rust Experts: Looking for Interview Study Participants*

Body:

Are you experienced at using unsafe? Do you face unique challenges in maintaining memory safety? We’d love to hear from you!

I’m a PhD Student at Carnegie Mellon University, and I’m running an interview study on unsafe Rust. I’m looking for people with at least one year of experience with Rust and who regularly use unsafe.

If that sounds like you, then we’d like you to complete the short questionnaire linked below. Eligible participants will be invited to participate in semi-structured interviews and will be paid \$30 per hour of interview time.

Thanks for reading!

[*insert link to survey*]

1.2.2 Community Survey Recruitment Template

Title: *We interviewed 19 Rust developers who regularly use unsafe. Now, we need your help to evaluate what we learned!*

Body:

Have you ever engaged with unsafe Rust? Please consider completing our community survey! All eligible participants who provide a link to their profile on either GitHub or the Rust Programming Language Forums with active account activity before [*insert start date*] will be entered into a drawing for one of two \$250 gift certificates to Amazon, Target, or Starbucks.

[*insert link to survey*]

I’m a PhD Student at Carnegie Mellon University, and I’m running a qualitative study on unsafe Rust. We reached out earlier this year [*insert link to previous post*] and interviewed 19 Rust developers who “*regularly write or edit*” unsafe code. This community survey targets a broader population and combines themes we learned from our interviews and related qualitative research. It should take 20 minutes to complete.

Thanks for reading!

[*insert link to survey*]

2 Consent

2.1 Screening & Community Surveys

The following consent form was displayed for both the screening and community surveys. Paragraphs annotated with “Screening Survey” and “Community Survey” were only displayed for the indicated survey.

This survey is part of a research study conducted by Ian McCormack at Carnegie Mellon University and is funded by the National Science Foundation.

Summary

[*Screening Survey*] As a participant, you will be asked to complete a 10-minute online survey on your use of Rust and its unsafe language features. Based on your responses, you may be invited to participate in a semi-structured, virtual interview on the same topics.

[*Community Survey*] As a participant, you will be asked to complete a 20-minute online survey on your experiences with Rust and its unsafe language features.

Purpose

[*Screening Survey*] The purpose of this research is to identify the use cases for Rust’s unsafe features and the challenges that developers face in writing unsafe code.

[*Community Survey*] The purpose of this research is to identify the use cases for Rust’s unsafe features and the challenges that developers face in writing unsafe code—as seen by the broader Rust community. Responses to this survey will help us evaluate the themes we observe in interviews from an earlier phase of this study.

Procedures

[*Screening Survey*] Data will be collected through an online survey consisting of short answer and multiple choice questions. Eligible participants may be invited to participate in semi-structured interviews based on their responses to the survey questions.

[*Community Survey*] Data will be collected through an online survey consisting of short answer and multiple choice questions.

Participant Requirements

[*Screening Survey*] Participation in this study is limited to individuals age 18 and older. There are no other requirements for participation in the survey. However, only participants that have at least 1 year of experience writing Rust and who regularly write code within unsafe blocks are eligible to be chosen for the interview stage.

[*Community Survey*] Participation in this study is limited to individuals age 18 and older who have at least 1 year of experience with Rust and have engaged with `unsafe` code.

Risks

The risks and discomfort associated with participation in this study are no greater than those ordinarily encountered in daily life or during other online activities.

Benefits

The results from this study may increase awareness about the challenges that participants face writing unsafe Rust and will inform the creation of development tools to assist with these challenges.

Compensation & Costs

[*Screening Survey*] There is no compensation for completing the survey. However, eligible participants

may be invited to participate in semi-structured, virtual interviews. If chosen, these participants will be compensated \$30 per hour of interview time, with a minimum compensation of \$30. This will be distributed as a gift card to the participant's choice of Target, Starbucks, or Amazon. There will be no cost to you if you participate in this study.

[*Community Survey*] You may optionally provide a link to your profile on either GitHub or the Rust Programming Language Forums, your name, and your email address to be entered into a drawing for one of two \$250 gift cards to either Amazon, Target, or Starbucks. Only users with accounts that show activity prior to the start date of the survey will be eligible for compensation. There will be no cost to you if you participate in this study.

Future Use of Information

In the future, once we have removed all identifiable information from your data (information), we may use the data for our future research studies, or we may distribute the data to other researchers for their research studies. We would do this without getting additional informed consent from you. Sharing of data with other researchers will only be done in such a manner that you will not be identified.

Confidentiality

[*Screening Survey*] If participants are eligible for participation in the interview stage, their name, email address, and affiliation will be collected at the end of the survey. Participants' affiliations may be published under broad classifiers including but not limited to "industry," "academia," and "open source."

[*Community Survey*] If you indicate that you are open to being contacted for follow-up interviews or surveys, then we will collect your name and email address. If you wish to be entered into the drawing for gift cards, then you must provide your name, email address, and a link to your profile on either GitHub or the Rust Programming Language Forums with account activity prior to the start date of the survey. If you participated in our prior interview study, you may choose to have your survey responses associated with your interview study data. In this case, we will ask for consent to collect your name and email address and to contact you if these do not match what you provided in the screening survey for the interview study. These identifiers will only be used for contacting participants for these purposes and will not be published.

By participating in this research, you understand and agree that Carnegie Mellon may be required to disclose your consent form, data and other personally identifiable information as required by law, regulation, subpoena or court order. Otherwise, your confidentiality will be maintained in the following manner:

Your data and consent form will be kept separate. Your consent form will be stored in a secure location on Carnegie Mellon property and will not be disclosed to third parties. By participating, you understand and agree that the data and information gathered during this study may be used by Carnegie Mellon and published and/or disclosed by Carnegie Mellon to others outside of Carnegie Mellon. However, your name, address, contact information and other direct personal identifiers will not be mentioned in any such publication or dissemination of the research data and/or results by Carnegie Mellon. Note that per regulation all research data must be kept for a minimum of 3 years.

Anonymized research data may be made accessible at any point to the National Science Foundation.

Right to Ask Questions & Contact Information If you have any questions about this study, you should feel free to ask them by contacting the Principal Investigator, Ian McCormack, Carnegie Mellon University, S3D, by email at icmccorm@cs.cmu.edu, or by phone: 651-239-0832. If you have questions later, desire additional information, or wish to withdraw your participation please contact the Principal Investigator by mail, phone or e-mail in accordance with the contact information listed above.

If you have questions pertaining to your rights as a research participant; or to report concerns to this study, you should contact the Office of Research integrity and Compliance at Carnegie Mellon University. Email: irb-review@andrew.cmu.edu. Phone: 412-268-4721.

Voluntary Participation Your participation in this research is voluntary. You may discontinue participation at any time during the research activity. You may print a copy of this consent form for your records.

2.2 Interviews

The following consent script was read aloud to interview participants. We only proceeded with interviews if participants answered “Yes” to each question.

The purpose of the research is to document the challenges developers face when writing unsafe Rust. Your responses to the screening survey and to this interview will be used to improve existing development tools and to create new ones that can meet these challenges. The study includes this interview and any follow-up interviews that occur. All interviews will be conducted remotely over Zoom.

You will be asked several open-ended questions about your use of Rust, focusing on the unsafe features of the language.

The information you provide to investigators may be exposed in the event of a confidentiality breach. All other discomfort and risks associated with participation in this interview are no greater than those ordinarily encountered in daily life or during other online activities.

You will be compensated with a \$30 gift card to your choice of Amazon, Starbucks, or Target.

Your name, affiliation, and email address will only be accessible to the investigators and will remain confidential unless we are legally mandated to disclose this information. We may publish all other screening survey responses and a written transcript of this interview without obtaining additional consent. Raw audio will not be published, but it and all other data will be retained by Carnegie Mellon for the required minimum of 3 years.

We will use automated transcription tools to produce a written transcript of your interview audio.

If you have any questions or concerns about this study, do not hesitate to voice them at any point with us or Carnegie Mellon’s Office of Research Integrity and Compliance. Your participation in this research is voluntary, and you may discontinue participation at any time during the interview.

Are you 18 years or older?

- Yes
- No

Have you understood the information?

- Yes
- No

Do you want to participate in this research and continue with the interview?

- Yes
- No

3 Interviews

3.1 Screening Survey

3.1.1 Eligibility

1. [*Required for eligibility, yes/no*] Do you have more than 1 year of experience using Rust?
2. [*Required for eligibility, yes/no*] Do you regularly reason about the correctness of unsafe code?
3. [*Required for eligibility if no to previous, yes/no*] Do you regularly write new Rust code or edit existing Rust code within an `unsafe` block or a function?

3.1.2 Background

1. [*Required ≥ 1 for eligibility, short answer*] How many years of experience do you have using Rust?
2. [*yes/no*] Do you call foreign functions from Rust
[*if yes*]
 - 2.1. [*check all languages that apply*] Foreign functions that you call **from** Rust are written in:
3. [*yes/no*] Do you call Rust functions from other languages?
[*if yes*]
 - 3.1. [*check all languages that apply*] Your code that calls **foreign Rust** functions is written in:
4. [*if yes to 2 or 3, select all that apply*] Select the tools and methods you use to create foreign bindings.
 - Manually written, bindgen, cbindgen, wasm, bindgen, cxx, autocxx, PyO3, hs-bindgen, Other (short answer)
5. [*yes/no*] Do you use Rust’s intrinsics?
6. [*yes/no*] Do you use system calls?
7. [*select all that apply*] Which of the following reference and memory container types have you converted to raw pointers or used to contain raw pointers?
 - `Box`, `Rc`, `UnsafeCell`, `Cell`, `Arc`, `RefCell`, `MaybeUninit`, `ManuallyDrop`, Other (short answer)
8. [*select all that apply*] Which of the following bug-finding tools do you use with codebases containing `unsafe` Rust?
 - Clippy, Miri, AddressSanitizer (ASAN), ThreadSanitizer (TSAN), UndefinedBehaviorSanitizer (UBSAN), MemorySanitizer (MSAN), Other (short answer)

3.1.3 Personal Information

The following questions request identifying information. Your affiliation will be collected so that we can determine if experiences with unsafe Rust vary based on the institutional context where development takes place. The specifics of your affiliation will not be published. Your affiliation will only be referred to in published data under broad classifications similar but not limited to “industry,” “academia,” and “open-source.” Your name and email are collected for scheduling and payment purposes.

1. [*short answer*] What is your name?
2. [*short answer*] What is your email?
3. [*short answer*] What is your current affiliation?

3.2 Interview Protocol

1. What have been your motivations for learning Rust and choosing to use it?
2. What do you use `unsafe` Rust for?
3. You mentioned using [*list types from screening*] in `unsafe` contexts. What do you use them for? Describe how you reason about memory safety in these situations.
4. You mentioned using [*list methods from screening*] to create foreign bindings. Describe your experiences with these methods.
5. How do you navigate the differences between Rust's memory model and other memory models?
6. Describe a bug you faced that involved `unsafe` Rust.
7. You mentioned using [*list tools from screening*]. Describe your experience using each one.
8. Do your development tools handle all of the problems that you face writing `unsafe` Rust?

4 Community Survey

4.1 Affirming Consent

1. [*required, yes/no*] I am age 18 or older.
2. [*required, yes/no*] I have read and understand the information above.
3. [*required, yes/no*] I want to participate in this research and continue with the survey.

4.2 Eligibility

1. [*required, ≥ 1*] How many years of experience do you have using Rust?
2. [*required, yes/no*] Have you written, edited, read, audited, or engaged with unsafe Rust code in any way?

4.3 Background

1. [*short answer*] How many years have you engaged with programming or the field of software engineering at-large? This includes acting in a professional capacity and as a hobby.
2. [*short answer*] How many years of experience do you have using C?
3. [*short answer*] How many years of experience do you have using C++?
4. [*select one*] How frequently do you engage with unsafe Rust code in any way?
 - (20) - *Daily*
 - (45) - *Weekly*
 - (58) - *Monthly*
 - (29) - *Yearly*
 - (8) - *Less than once a year*
5. Have you ever committed unsafe Rust code to a public GitHub repository?
 - (118) - *Yes*
 - (42) - *No*
6. Do you regularly write new Rust code or edit existing Rust code within an unsafe block or function?
 - (85) - *Yes*
 - (75) - *No*
7. [*select one*] How frequently do you write new Rust code or edit existing Rust code within an unsafe block or function?
 - (12) - *Daily*
 - (29) - *Weekly*
 - (69) - *Monthly*
 - (33) - *Yearly*
 - (17) - *Less than once a year*

4.4 Domain

1. [*select all*] Which of the following types of applications have you developed or contributed to in any capacity using any programming language?

- (120) - *Web applications - Backend*
- (111) - *Data structures and language-level primitives*
- (103) - *Serialization & deserialization*
- (96) - *Networking & distributed systems*
- (93) - *Embedded systems*
- (90) - *Web applications - Frontend*
- (85) - *Interpreters & runtime systems*
- (83) - *Image & Data processing*
- (68) - *Game development*
- (60) - *Operating systems*
- (50) - *Database systems*
- (46) - *Non-JIT Compilation*
- (34) - *Static analysis*
- (21) - *Other*
- (13) - *Manufacturing*
- (11) - *JIT Compilation*
- (9) - *Web browsers*

2. [*select all*] Which of the following types of applications have you developed or contributed in any capacity using Rust?

- (85) - *Data structures and language-level primitives*
- (85) - *Web applications - Backend*
- (83) - *Serialization & deserialization*
- (66) - *Networking & distributed systems*
- (57) - *Embedded systems*
- (55) - *Image & Data processing*
- (49) - *Interpreters & runtime systems*
- (46) - *Game development*
- (31) - *Operating systems*
- (31) - *Web applications - Frontend*
- (30) - *Database systems*
- (27) - *Non-JIT Compilation*
- (21) - *Static analysis*
- (14) - *Other*
- (6) - *JIT Compilation*
- (5) - *Manufacturing*
- (2) - *Web browsers*

3. Have you rewritten part or all of an application in Rust that was originally written in another language?

- (115) - *Yes*
- (45) - *No*

4. Have you authored or regularly contributed to a Rust crate that is published on crates.io?

- (106) - *Yes*
- (54) - *No*

5. Have you authored or regularly contributed to a Rust crate not published on crates.io?

- (129) - *Yes*
- (31) - *No*

6. [*select one*] When you first started using Rust, was it easy or difficult to write code that passed the borrow checker?

- (6) - *Extremely difficult*
- (75) - *Somewhat difficult*
- (36) - *Neither easy nor difficult*
- (34) - *Somewhat easy*
- (9) - *Extremely easy*

4.5 Unsafe Features

1. [*select all*] Which of the following Rust types have you used in an unsafe context for any purpose?

- (110) - *MaybeUninit*
 - (81) - *Box*
 - (73) - *ManuallyDrop*
 - (72) - *UnsafeCell*
 - (50) - *Pin*
 - (48) - *Arc*
 - (46) - *Mutex*
 - (36) - *Cell*
 - (33) - *OnceCell*
 - (30) - *Rc*
 - (24) - *RefCell*
 - (18) - *Other*
 - (16) - *LazyCell*
2. [*select all*] Which of the following unsafe Rust features have you used?
- (142) - *Calling unsafe functions written in Rust*
 - (134) - *Dereferencing a raw pointer*
 - (118) - *mem::transmute*
 - (112) - *Calling foreign functions*
 - (93) - *Pointer arithmetic*
 - (84) - *Implementing an unsafe trait*
 - (71) - *Mutable static variables*
 - (67) - *Manual memory allocation*
 - (60) - *In-place initialization or updates*
 - (60) - *System calls*
 - (56) - *Inline assembly*
 - (54) - *Declaring an unsafe trait*
 - (53) - *Accessing the fields of a union type*
 - (47) - *Implementing a custom allocator*
 - (43) - *Intrinsics*
 - (40) - *Self-referential structs*
 - (26) - *Global assembly*
 - (14) - *Cyclic aliasing patterns*
 - (7) - *Other*
3. Have you contributed to a Rust crate that is designed to be used from at least one other language?
- (76) - *Yes*
 - (84) - *No*

4.6 UnsafeCell

Questions in this section were only shown to respondents if they had selected “**UnsafeCell**” in their response to Question 4.5.0.1 of the survey.

1. [*select one*] When you choose to use **UnsafeCell**, how often are you certain that it is necessary to avoid undefined behavior?
- (0) - *Never*
 - (4) - *About half the time*
 - (7) - *Sometimes*
 - (30) - *Most of the time*
 - (31) - *Always*

4.7 Calling Unsafe Functions

This section was only shown to respondents if they had selected “Calling unsafe functions written in Rust” or “Calling foreign functions” in their response to Question 4.5.0.1 of the survey.

1. [*select one*] When you use an **unsafe** API, how often do you look for documentation to ensure that you meet all of its requirements for safety and correctness?
- (2) - *Never*
 - (10) - *Sometimes*
 - (11) - *About half the time*

- (34) - *Most of the time*
 - (95) - *Always*
2. [*select one*] When you use an `unsafe` API, how often do you insert runtime checks to ensure that you meet its requirements for safety and correctness?
- (13) - *Never*
 - (54) - *Sometimes*
 - (21) - *About half the time*
 - (44) - *Most of the time*
 - (20) - *Always*

4.8 Intrinsic

This section was only shown to respondents if they had selected “Intrinsic” in their response to Question 4.5.0.1 of the survey.

1. [*select all*] Which of the following types of intrinsic have you used in Rust applications?
- (37) - *Architecture-specific or SIMD intrinsic*
 - (18) - *Atomics*
 - (14) - *Const intrinsic*
 - (7) - *Volatiles*
 - (6) - *Other*
2. Do you use runtime checks in Rust applications, such as the `is_x86_feature_detected!` macro, to determine when certain hardware-specific features are supported?
- (18) - *Yes*
 - (25) - *No*

4.9 ManuallyDrop + MaybeUninit

This section was only shown to respondents if they had selected “MaybeUninit” and “ManuallyDrop” in their response to Question 4.5.0.2 of the survey.

1. [*select one*] In Rust, how often is your use of `ManuallyDrop` associated with your use of `MaybeUninit`?
- (15) - *Never*
 - (29) - *Sometimes*
 - (7) - *Unsure*
 - (2) - *About half the time*
 - (13) - *Most of the time*
 - (1) - *Always*

4.10 Box

This section was only shown to respondents if they had selected “Box” in their response to Question 4.5.0.2 of the survey.

1. According to Rust’s current semantics, if you re-wrap a raw pointer as a `Box` using `Box::from_raw`, and then you move the `Box`, the raw pointer becomes invalid. Has this caused problems or architectural challenges in the applications that you have contributed to?
 - (14) - *Yes*
 - (67) - *No*
2. Have you ever encountered memory leaks due to leaking a raw pointer using `Box::into_raw()` and then forgetting to re-wrap it using `Box::from_raw()`?
 - (18) - *Yes*
 - (63) - *No*

4.11 Why unsafe?

1. [*select all*] Think about the situations where you have typically used `unsafe`. Which of the following reasons have motivated you to do so?
 - (123) - *I am not aware of a safe alternative at any level of ease-of-use or performance.*
 - (75) - *I could use a safe pattern but unsafe is faster or more space-efficient.*
 - (29) - *I could use a safe pattern but unsafe is easier to implement or more ergonomic.*
 - (19) - *Other*

4.12 Why unsafe? - No clear alternative

This section was only shown to respondents if they had selected “I am not aware of a safe alternative at any level of ease-of-use or performance” in their response to Question 4.11.0.1 of the survey.

1. [*select all*] Which of the following `unsafe` features have you used because you were not aware of a safe alternative?

• (80) - <i>Calling unsafe functions written in Rust</i>	• (23) - <i>Accessing the fields of a union type</i>
• (76) - <i>Calling foreign functions</i>	• (23) - <i>In-place initialization or updates</i>
• (72) - <i>Dereferencing a raw pointer</i>	• (23) - <i>Intrinsics</i>
• (62) - <i>mem::transmute</i>	• (23) - <i>Manual memory allocation</i>
• (49) - <i>Implementing an unsafe trait</i>	• (23) - <i>Mutable static variables</i>
• (40) - <i>Pointer arithmetic</i>	• (20) - <i>Implementing a custom allocator</i>
• (35) - <i>Inline assembly</i>	• (19) - <i>Self-referential structs</i>
• (31) - <i>System calls</i>	• (12) - <i>Global assembly</i>
• (26) - <i>Declaring an unsafe trait</i>	• (6) - <i>Cyclic aliasing patterns</i>
	• (2) - <i>Other</i>
2. [*select one*] When you feel that you have no clear alternative other than using `unsafe`, how often are you certain that it would be completely impossible to accomplish this task using a safe design pattern?
 - (2) - *Never*
 - (19) - *Sometimes*
 - (4) - *Unsure*

- (18) - *About half the time*
- (63) - *Most of the time*
- (17) - *Always*

The next question was only shown to respondents if they had selected any of the following types in their response to Question 4.5.0.2 of the survey.

1. [*select all*] Which of the following Rust types have you used because you were not aware of a safe alternative?
 - (60) - *MaybeUninit*
 - (40) - *UnsafeCell*
 - (36) - *ManuallyDrop*
 - (3) - *Other*

4.13 Why unsafe? - Ergonomics

This section was only shown to respondents if they had selected “I could use a safe pattern but unsafe is easier to implement or more ergonomic” in their response to Question 4.11.0.1 of the survey.

1. [*select all*] Which of the following **unsafe** features have you used because it was easier to implement or more ergonomic than a safe alternative?

• (17) - <i>mem::transmute</i>	• (5) - <i>Self-referential structs</i>
• (13) - <i>Calling unsafe functions written in Rust</i>	• (4) - <i>Declaring an unsafe trait</i>
• (13) - <i>Dereferencing a raw pointer</i>	• (4) - <i>Implementing a custom allocator</i>
• (10) - <i>Calling foreign functions</i>	• (3) - <i>Accessing the fields of a union type</i>
• (10) - <i>Mutable static variables</i>	• (3) - <i>Intrinsics</i>
• (9) - <i>Pointer arithmetic</i>	• (3) - <i>System calls</i>
• (6) - <i>Implementing an unsafe trait</i>	• (2) - <i>Cyclic aliasing patterns</i>
• (6) - <i>Manual memory allocation</i>	• (2) - <i>Global assembly</i>
• (5) - <i>In-place initialization or updates</i>	• (2) - <i>Inline assembly</i>
	• (0) - <i>Other</i>

This question was only shown to respondents if they had selected any of the following types in their response to Question 4.5.0.2 of the survey.

2. [*select all*] Which of the following Rust types have you used because it was easier to implement or more ergonomic than a safe alternative?
 - (10) - *ManuallyDrop*
 - (10) - *MaybeUninit*
 - (7) - *UnsafeCell*
 - (1) - *Other*

4.14 Why unsafe? - Performance

This section was only shown to respondents if they had selected “I could use a safe pattern but unsafe is easier to implement or more ergonomic” in their response to Question 4.11.0.1 of the survey.

1. [*select all*] Which of the following unsafe features have you used because it performed faster or was more space-efficient than a safe alternative?

- (41) - *mem::transmute*
- (40) - *Calling unsafe functions written in Rust*
- (40) - *Dereferencing a raw pointer*
- (31) - *In-place initialization or updates*
- (27) - *Pointer arithmetic*
- (20) - *Intrinsics*
- (19) - *Manual memory allocation*
- (18) - *Inline assembly*
- (16) - *Calling foreign functions*
- (16) - *Mutable static variables*
- (16) - *Self-referential structs*
- (15) - *Implementing an unsafe trait*
- (13) - *Accessing the fields of a union type*
- (10) - *Declaring an unsafe trait*
- (9) - *Implementing a custom allocator*
- (8) - *Cyclic aliasing patterns*
- (8) - *System calls*
- (4) - *Global assembly*
- (2) - *Other*

The next question was only shown to respondents if they had selected any of the following types in their response to Question 4.5.0.2 of the survey.

2. [*select all*] Which of the following Rust types have you used because it performed faster or was more space efficient than a safe alternative?
 - (46) - *MaybeUninit*
 - (22) - *UnsafeCell*
 - (18) - *ManuallyDrop*
 - (2) - *Other*
3. [*select one*] When you choose to use unsafe because it performs faster or is more space efficient, how often do you measure the difference?
 - (8) - *Never*
 - (19) - *Sometimes*
 - (14) - *About half the time*
 - (16) - *Most of the time*
 - (19) - *Always*
4. [*select one*] When you use unsafe because it performs faster or is more space efficient, are you typically introducing small-scale optimizations into existing applications, or are you designing large-scale abstractions that are purpose-built for performance?
 - (32) - *Both*
 - (20) - *Small-scale optimizations*
 - (15) - *Large-scale components purpose-built for performance.*
 - (5) - *Unsure*
 - (4) - *Other*

4.15 Performance VS Safety

1. [*select one*] You are writing an arbitrary Rust application, and you have the opportunity to increase its speed or space-efficiency using unsafe code. You are certain that the unsafe code is sound, and your test cases pass Miri, but the performance increase is marginal. Which would you choose?
 - (110) - *Use safe*
 - (28) - *Unsure*
 - (22) - *Use unsafe*

The next question was only shown to participants if they had answered “Yes” to either Question 4.4.0.3 or Question 4.4.0.4.

2. [*select one*] You are writing a Rust crate and you have the opportunity to increase its speed or space-efficiency using unsafe code. You are certain that the unsafe code is sound, and your test cases pass Miri, but the performance increase is marginal. Which would you choose:
 - (104) - *Use safe*
 - (21) - *Unsure*
 - (19) - *Use unsafe*

4.16 Local Minimal Unsafe

1. [*select one*] If another Rust developer at your skill level selected a random `unsafe` block or function from code that you have written, would it be easy or difficult for them to understand?
 - (2) - *Extremely difficult*
 - (22) - *Somewhat difficult*
 - (38) - *Neither easy nor difficult*
 - (82) - *Somewhat easy*
 - (16) - *Extremely easy*
2. [*select all*] Select each of the following features or attributes that you have chosen to enable in a Rust module.
 - (47) - *unsafe_block_in_unsafe_fn*
 - (38) - *forbid(unsafe_code)*
 - (37) - *deny(unsafe_code)*
 - (8) - *allow(improper_ctypes)*
 - (4) - *const_raw_ptr_deref*
 - (2) - *allow(improper_ctypes_definitions)*
3. [*select one*] How often do you refactor Rust applications to remove unsafe code?
 - (40) - *Never*
 - (101) - *Sometimes*
 - (10) - *About half the time*
 - (8) - *Most of the time*
 - (1) - *Always*
4. [*select one*] How often do you choose to avoid using an unsafe API when a safe alternative exists?
 - (2) - *Never*
 - (14) - *Sometimes*
 - (12) - *About half the time*
 - (98) - *Most of the time*
 - (34) - *Always*

4.17 Encapsulation

1. Have you exposed an unsafe API to other Rust developers?
 - (69) - *Yes*
 - (91) - *No*
2. Have you exposed a safe API to other Rust developers that encapsulates unsafe code?
 - (141) - *Yes*
 - (19) - *No*

4.18 Encapsulation - Unsafe API

This section was only shown to respondents if they had selected “Yes” in their response to Question 4.17.0.1.

1. [*select all*] Which of the following reasons have motivated you to expose an unsafe API to users?
 - (51) - *Impossible to encapsulate without imposing safety requirements on the user*
 - (34) - *To provide a more performant equivalent of a safe API*
 - (9) - *To provide a more ergonomic equivalent of a safe API*
 - (5) - *Other*
2. [*select one*] When you expose an unsafe API, how often are its users responsible for meeting certain requirements for correctness and safety?
 - (1) - *Never*
 - (3) - *Sometimes*
 - (1) - *Unsure*
 - (4) - *About half the time*
 - (18) - *Most of the time*
 - (42) - *Always*

4.19 Encapsulation - Unsafe API - Invariants

Questions in this section were only shown to respondents if they had not selected “Never” in their response to Question 4.18.0.2.

1. [*select one*] When you expose an unsafe API to users, and it is their responsibility to ensure that certain requirements are met, how often do you document these requirements?
 - (0) - *Never*
 - (2) - *Sometimes*
 - (5) - *About half the time*
 - (8) - *Most of the time*
 - (53) - *Always*
2. [*select one*] When you expose an unsafe API to other users, and it is their responsibility to ensure that certain requirements are met, how often do you include runtime checks for these requirements?
 - (11) - *Never*
 - (37) - *Sometimes*
 - (8) - *About half the time*
 - (10) - *Most of the time*
 - (2) - *Always*

4.20 Encapsulation - Safe API

This section was only shown to respondents if they had selected “Yes” in their response to Question 4.17.0.2.

1. [*select one*] When you expose a safe API for unsafe code, how often do you include runtime checks to ensure that its requirements for correctness and safety are met?
 - (11) - *Never*
 - (30) - *Sometimes*
 - (18) - *About half the time*
 - (47) - *Most of the time*
 - (35) - *Always*
2. [*select one*] When you expose a safe API for unsafe code, how often are all of its requirements for correctness and safety satisfied by the properties of Rust’s type system?
 - (1) - *Never*
 - (25) - *Sometimes*
 - (11) - *Unsure*
 - (16) - *About half the time*
 - (55) - *Most of the time*
 - (33) - *Always*

4.21 FFI - Background

This section was only shown to respondents if they had selected “Calling foreign functions” in their response to Question 4.5.0.2.

1. [*select all*] When you call foreign functions from Rust or write Rust bindings to foreign functions, which languages are these functions written in?
 - (101) - *C*
 - (64) - *C++*
 - (24) - *Assembly*
 - (19) - *Python*
 - (15) - *JavaScript*
 - (12) - *Other*
 - (10) - *TypeScript*
 - (8) - *Java*
 - (4) - *Go*
 - (4) - *Swift*
 - (3) - *C#*
 - (3) - *Dart*
 - (1) - *Haskell*
 - (1) - *OCaml*
 - (1) - *Ruby*
 - (0) - *Erlang*
 - (0) - *Futhark*
 - (0) - *Julia*
2. [*select one*] Do you write or generate bindings to foreign functions?
 - (60) - *I write bindings manually and generate bindings using a tool*
 - (29) - *I generate bindings using a tool*
 - (15) - *I write bindings manually*
 - (8) - *I do not write or generate bindings*
3. [*select one*] Do you trust FFI bindings that are written by hand?
 - (3) - *Definitely not*
 - (5) - *Probably not*

- (54) - *Might or might not*
- (40) - *Probably yes*
- (10) - *Definitely yes*

4. [*select one*] Do you trust FFI bindings that are generated by a tool?

- (0) - *Definitely not*
- (2) - *Probably not*
- (22) - *Might or might not*
- (68) - *Probably yes*
- (20) - *Definitely yes*

5. [*select one*] Have you ever encountered incorrect FFI bindings in a Rust application?

- (48) - *No*
- (39) - *Yes*
- (25) - *Unsure*

This question was only shown to respondents if they had selected “Yes” to the previous question.

6. [*select one*] Have these incorrect bindings typically been written manually, generated by a tool, or both?

- (14) - *Both*
- (13) - *Written manually*
- (5) - *Generated by a tool*
- (7) - *Unsure*

4.22 FFI - Memory Model

This section was only shown to respondents if they had selected “Calling foreign functions” in their response to Question 4.5.0.2.

The next question was only shown to respondents if they had selected “Box”, “Arc”, or “Rc” in their response to Question 4.5.0.1.

1. [*select all*] Select each of the following memory container types that you have used to allocate memory for a foreign function call. This process includes creating the container, exposing it as a raw pointer using `into_raw()`, and then passing it into a foreign function.

- (58) - *Box*
- (17) - *Arc*
- (9) - *Other*
- (5) - *Rc*

2. [*select one*] When you receive a raw pointer to memory allocated by an FFI call, how often do you store it as an `UnsafeCell`?

- (72) - *Never*
- (25) - *Sometimes*
- (11) - *Most of the time*
- (4) - *About half the time*
- (0) - *Always*

3. [*select one*] Do you pass Rust’s abstract data types (structs, enums) by value across FFI boundaries?
 - (65) - *No*
 - (31) - *Yes*
 - (16) - *Unsure*
4. [*select one*] Do you convert raw pointers to memory allocated by FFI calls into safe references, such as `&T` or `&mut T`?
 - (32) - *No*
 - (65) - *Yes*
 - (15) - *Unsure*
5. [*select one*] How often do you intentionally avoid passing Rust’s abstract data types (structs, enums) by value across FFI boundaries?
 - (5) - *Never*
 - (22) - *Sometimes*
 - (22) - *Unsure*
 - (8) - *About half the time*
 - (31) - *Most of the time*
 - (24) - *Always*
6. [*select one*] How often do you intentionally avoid converting raw pointers to memory allocated by FFI calls into safe references, such as `&T` or `&mut T`?
 - (8) - *Never*
 - (40) - *Sometimes*
 - (26) - *Unsure*
 - (12) - *About half the time*
 - (14) - *Most of the time*
 - (12) - *Always*

4.23 FFI - Bindings - Automation

This section was only shown to respondents if they had selected “I generate bindings using a tool” or “I write bindings manually and generate bindings using a tool” in their response to Question 4.21.0.2.

1. [*select all*] Which of the following binding generation tools have you used?

• (71) - <i>bindgen</i> • (42) - <i>wasm-bindgen</i> • (37) - <i>cbindgen</i> • (35) - <i>PyO3</i> • (29) - <i>CXX</i>	• (11) - <i>Emscripten</i> • (10) - <i>Other</i> • (1) - <i>Diplomat</i> • (1) - <i>UniFFI</i>
--	---

The next question was only shown to participants if they had selected “bindgen” in their response to Question 4.23.0.1

2. [*select one*] Do you typically generate bindings locally and check them in, or do you generate them as a CI step? (*we only have partial data for this question; it was added after the survey began in response to community feedback*).

- (36) - *Locally*
- (12) - *Other*
- (8) - *In CI*

The next question was only shown to participants if they had selected “wasm-bindgen” in their response to Question 4.23.0.1

3. [*select one*] Has it typically been difficult or easy to configure wasm-bindgen?

- (1) - *Extremely difficult*
- (7) - *Somewhat difficult*
- (12) - *Neither easy nor difficult*
- (18) - *Somewhat easy*
- (4) - *Extremely easy*

4.24 Validation

1. [*select all*] Which of the following formal methods tools have you used with Rust?

- (5) - *Kani*
- (3) - *Creusot*
- (3) - *Prusti*
- (1) - *Flux*
- (0) - *Verus*
- (1) - *Other*

2. [*select all*] Which of the following sanitizers, fuzzers, and dynamic analysis tools have you used for Rust applications?

- | | |
|--|---|
| • (98) - <i>Miri</i> | • (14) - <i>Loom</i> |
| • (70) - <i>Valgrind</i> | • (12) - <i>MemorySanitizer (MSAN)</i> |
| • (42) - <i>cargo fuzz</i> | • (4) - <i>Other</i> |
| • (41) - <i>AddressSanitizer (ASAN)</i> | • (2) - <i>Shuttle</i> |
| • (21) - <i>ThreadSanitizer (TSAN)</i> | • (1) - <i>HWAddressSanitizer (HWASAN)</i> |
| • (20) - <i>UndefinedBehaviorSanitizer (UBSAN)</i> | • (0) - <i>MemTagSanitizer</i> |
| • (17) - <i>libFuzzer</i> | • (0) - <i>KernelControlFlowIntegrity</i> |
| • (14) - <i>LeakSanitizer (LSAN)</i> | • (0) - <i>ShadowCallStack</i> |

3. [*select one*] How frequently do you use a debugger with a Rust application?

- (22) - *Never*
- (24) - *Less than once a year*
- (24) - *Yearly*
- (46) - *Monthly*
- (28) - *Weekly*
- (16) - *Daily*

4. [*select all*] Which of the following types of bugs and undefined behaviors have you encountered in a Rust application and were caused by unsafe code?

- (67) - *Out-of-bounds access*
 - (54) - *Using uninitialized memory*
 - (50) - *Null pointer dereference*
 - (49) - *Memory leak*
 - (49) - *Use after free*
 - (43) - *Unaligned memory access*
 - (40) - *Violation of stacked or tree borrows*
 - (34) - *Double free*
 - (31) - *Incorrectly declared bindings to intrinsics or foreign functions.*
 - (27) - *External runtime or hardware interface does not behave according to its written specification.*
 - (20) - *Failure to meet an application's functional requirements.*
 - (20) - *Unwinding across foreign stack frames without using an -unwind ABI*
 - (16) - *Integer overflow*
 - (14) - *Deallocating memory with an incorrect Layout*
 - (10) - *Other*
 - (7) - *Creating a zero-size allocation with an allocator that requires nonzero size*
5. [*select one*] How often do you write tests for Rust applications that use unsafe?
- (9) - *Never*
 - (39) - *Sometimes*
 - (15) - *About half the time*
 - (46) - *Most of the time*
 - (51) - *Always*
6. [*select one*] How often do you audit your dependencies' use of unsafe?
- (77) - *Never*
 - (64) - *Sometimes*
 - (9) - *About half the time*
 - (7) - *Most of the time*
 - (3) - *Always*
7. [*select one*] Select each of the following tools that you use to audit your dependencies.
- (51) - *cargo-audit*
 - (34) - *cargo-update*
 - (25) - *cargo-deny*
 - (18) - *cargo-geiger*
 - (6) - *cargo-vet*
8. [*select one*] One of your options for a dependency has an unsafe API. Are you likely or unlikely to avoid choosing this dependency if other options have a safe API?
- (9) - *Extremely unlikely*
 - (20) - *Somewhat unlikely*
 - (52) - *Neither likely nor unlikely*
 - (51) - *Somewhat likely*
 - (28) - *Extremely likely*

4.25 Validation - Miri

Questions in this section were only shown to participants if they had selected “Miri” in their response to Question 4.24.0.2.

1. [*select all*] Which of the following issues have deterred you from using Miri to test a Rust application?
 - (42) - *Lack of support for foreign function calls*
 - (25) - *Slow performance*
 - (19) - *Lack of support for inline assembly*
 - (9) - *Other*
 - (4) - *My unsafe code is in violation of Stacked Borrows*
 - (3) - *My unsafe code is in violation of Tree Borrows*
2. [*select one*] How often do you run test cases in Miri?
 - (13) - *Never*
 - (42) - *Sometimes*
 - (6) - *About half the time*
 - (24) - *Most of the time*
 - (13) - *Always*
3. [*select all*] Which of the following bugs has Miri detected in applications that you have contributed to?

• (41) - <i>Stacked Borrows Violation</i>	• (19) - <i>Memory leak</i>
• (32) - <i>Using uninitialized memory</i>	• (16) - <i>Tree Borrows Violation</i>
• (24) - <i>Unaligned memory access</i>	• (15) - <i>Data race</i>
• (24) - <i>Use-after-free</i>	• (15) - <i>Null pointer dereference</i>
• (23) - <i>Out-of-bounds access</i>	• (5) - <i>Other</i>

The next question was only shown to participants if they had selected either “Stacked Borrows Violation” or “Tree Borrows Violation” in their answer to Question 4.25.0.3.

4. [*select one*] When Miri detects a borrowing violation in your code, has it usually been easy or difficult to fix?
 - (3) - *Extremely difficult*
 - (15) - *Somewhat difficult*
 - (11) - *Neither easy nor difficult*
 - (11) - *Somewhat easy*
 - (5) - *Extremely easy*

4.26 Community and Culture

1. [*select all*] Select each of the following online communities where you have posted asking for advice related to either undefined behavior or the correct use of unsafe code.
 - (35) - *GitHub*
 - (35) - *The Rust Community Discord*
 - (32) - *The Rust Programming Language Forums*
 - (30) - *Reddit (/r/rust)*

- (11) - *Other*
2. [*select one*] How do you perceive that the Rust community views unsafe code in general?
 - (11) - *Extremely negatively*
 - (92) - *Somewhat negatively*
 - (43) - *Neither positively nor negatively*
 - (7) - *Unsure*
 - (5) - *Somewhat positively*
 - (2) - *Extremely positively*
 3. [*select one*] How do you perceive that the Rust community views the type of unsafe code that you write?
 - (4) - *Extremely negatively*
 - (22) - *Somewhat negatively*
 - (49) - *Neither positively nor negatively*
 - (60) - *Unsure*
 - (23) - *Somewhat positively*
 - (2) - *Extremely positively*
 4. [*select one*] How often is the Rust community's guidance and documentation adequate for you to know how to use unsafe correctly?
 - (1) - *Never*
 - (24) - *Sometimes*
 - (12) - *Unsure*
 - (11) - *About half the time*
 - (102) - *Most of the time*
 - (10) - *Always*
 5. [*select one*] Do you agree or disagree with the following statement:"No documentation is better than incorrect documentation."
 - (8) - *Strongly disagree*
 - (19) - *Somewhat disagree*
 - (23) - *Neither agree nor disagree*
 - (61) - *Somewhat agree*
 - (49) - *Strongly agree*

4.27 Demographics

1. [*select one*] What is your age?
 - (90) - *18-29*
 - (43) - *30-39*
 - (15) - *40-49*
 - (7) - *50-59*
 - (0) - *60-69*
 - (0) - *Over 70*
 - (5) - *Prefer not to answer*
2. [*select one*] What is your highest completed level of education?

- (8) - *Some high school*
- (17) - *High school diploma/GED*
- (26) - *Some college*
- (57) - *Bachelor's degree*
- (34) - *Master's degree*
- (14) - *PhD*
- (4) - *Prefer not to answer*

3. [*select all*] What is your gender?

- (131) - *Man*
- (10) - *Non-binary*
- (4) - *Woman*
- (15) - *Prefer not to disclose*

4. [*select one*] Select each of the affiliations that best describes your role as a Rust developer.

- (31) - *Academia*
- (91) - *Industry*
- (115) - *Open-source contributor*
- (7) - *Rust project member*
- (0) - *Rust foundation member*
- (8) - *Other*

5. [*select all*] Where did you first hear about this survey?

- (83) - *Reddit (/r/rust)*
- (52) - *"This Week in Rust"*
- (16) - *The Rust Programming Language Forums*
- (10) - *The Rust Programming Language Community Discord*
- (0) - *X*
- (3) - *Other*

4.28 Personal Information - Consent

1. [*yes/no*] Do you wish to be entered into a drawing for a \$250 gift card to Amazon, Starbucks, or Target? To be eligible, you will need to provide a valid username from GitHub or the Rust Programming Language Forums and an email address. Your account must have activity prior to the start date of the survey.
2. [*yes/no*] Are you open to being contacted for follow-up surveys or interviews?
3. [*yes/no*] Did you participate in our initial interview study this spring?

4.29 Personal Information - Connection

Questions in this section were only shown to respondents who had selected "Yes" in their response to Question 4.28.0.3.

1. [*yes/no*] Would you be open to providing your name and email address so that we can associate your responses to this survey with your data from the interview study? By selecting 'Yes,' you consent to be contacted for clarification if the name and email address you provided do not match earlier data.

4.30 Personal Information - Drawing

Questions in this section were only shown to participants if they had selected “Yes” in their response to Question 4.28.0.1.

The online profile that you provide will only be used to prevent multiple survey responses from being submitted by the same user. It will not be published.

1. [*short answer*] Copy and paste a link to your profile page on either GitHub or the Rust Programming Language Forums.
2. [*select one*] Which type of gift card would you prefer?
 - *Amazon*
 - *Starbucks*
 - *Target*

4.31 Personal Information - Email

Questions in this section were only shown to participants if they had selected “Yes” in their response to Question 4.28.0.1, Question 4.28.0.3, or Question 4.29.0.1.

Your name and email are collected for the purpose of contacting you if you are eligible for compensation and are selected, if you consented to be contacted for follow-up surveys or interviews, or if you consented to associate your survey response with prior interview study data and the name and email you provide do not match what you previously provided. These identifiers will not be used for any other purposes and will not be published.

1. [*short answer*] What is your name?
2. [*short answer*] What is your email?

5 Interrater Reliability

We used interrater reliability as a mechanism for refining our codebook. This took place across 7 “rounds” between the first and second authors. During each round, we coded sets of quotes that were randomly selected from across all interviews. To sample quotes, we concatenated all of the interview transcripts into a single document and randomly sampled character indices. The first author visited each index and selected a snippet of text around that location. Each round had a unique set of quotes. When an index was randomly selected and it was near a quote used in a previous round, the first author would choose an adjacent quote or continue sampling to find a new one.

In each of the first 5 rounds, we coded 20 randomly selected quotes. Since our reliability ratings were inconsistent between each round, we switched to coding a larger sample of 60 quotes for rounds 6 and 7. After round 6, we excluded certain themes where establishing reliability was less crucial, since their constituent codes were either keywords in the transcripts (“Interoperation,” “Types,” “Tools”); corresponded to a singular, abstract theme (“Uncertainty”); or had not been encountered during previous rounds (“Challenges with Bug-finding”). The theme “**unsafe** data structure” was excluded from this round because its purpose was to identify data structures that interview participants had mentioned so that the authors could create prototypical examples for future research, so it was not relevant to our research questions. We also chose smaller quote sizes for round 7, since we found that many of our differences in coding decisions during the previous round were due having missed details instead of actively disagreeing about the meaning of a quote.

We calculated interrater reliability using Krippendorff’s alpha, and our scores for each round are shown in Table 1. Since rounds 1 through 5 consisted of small sets of quotes, we calculated a reliability score for all coding decisions across each of these rounds prior to moving on to round 6. A “-” indicates that a theme was excluded from a round, while a “?” indicates that a theme was included in that round, but that no quotes were coded for it. A “?” is equivalent to an alpha score of 1, but we found that this distinction was useful to determine if we had actually encountered quotes for that theme during a particular set of rounds.

During each round, coding decisions were recorded in an online form. The form was split into multiple sections, with one question per theme. Table 2 outlines the structure of the survey and provides references to tables containing the codes for each theme. Some themes, such as “Uncertainty,” are their own code and are can only be present or absent from a quote. Others, such as “Tools,” can have multiple codes appear within a single quote. After each round, the first and third author met and refined the themes, making changes to the form as necessary. Each of these changes is reflected in the structure of Table 2 and the tables it references. The replication package contains an unabbreviated list of the themes and codes used during each round.

Table 1: Interrater reliability between the first and second author calculated using Krippendorff’s alpha. The column “Survey Questions” includes references to sections or individual questions that have a direct correspondence to each theme, indicating that a theme is being used as a product of our qualitative analysis.

Theme	Rounds 1-5	Round 6	Round 7	Survey Questions
Stigma Against unsafe	1	0.47	?	4.26.0.2, 4.26.0.3
Issues With Miri	0.66	0.49	0.8	4.25.0.1
Documented Invariants	0.49	1	0	4.19.0.1
Uncertainty	0.86	0.66	-	-
Interoperation	0.83	0.68	-	4.21.0.1
Types	0.75	0.87	-	4.5.0.2
Binding Generation Preferences	0.74	0.35	0.5	4.21, 4.23
Domain	0.71	0.59	0.53	4.4.0.1, 4.4.0.2
Found a bug?	0.45	0.65	0.77	4.24.0.4, 4.25.0.3
Why unsafe ?	0.33	0.23	0.7	4.11
Tools	0.73	0.74	-	4.24.0.1, 4.24.0.2
Did they expose an unsafe API?	?	0	0	4.17.0.1
unsafe Operations	0.63	0.37	0.48	4.5.0.2
Challenges with Bug-finding	?	?	-	-
unsafe Data Structure	0.58	-0.01	-	-

Table 2: Themes and theme titles used during inter-rater reliability. The “Type” column indicates the method used to indicate that a code for that theme was present. Themes of type “Yes / No” were recorded by indicating their presence or absence by selecting “Yes,” “N/A”, or leaving the prompt unselected. Themes that are “Select One” or “Select All” have multiple constituent codes, where either one or all codes that apply could be selected, respectively. For themes with multiple codes, the “Codes” column includes a reference to a table of codes for that theme located further in the Appendix.

Theme	Codes	Type	Rounds	Title
Bug-finding	Table 3	Select One	1-6	Challenges with bug-finding
Bugs	Table 4	Select All	7	Did the participant encounter any of these bugs?
			1-6	Found a bug?
Documented Invariants	N/A	Yes / No	2-7	Did they document pre and post-conditions or invariants for safety of unsafe code?
			1	Documented pre and postconditions for safety?
Domain	Table 5	Select One	1-7	Domain
Generation vs. Validation	Table 6	Select One	1-6	Binding generation preferences
			7	Did the participant express a preference for a certain type of binding generation?
Interoperation		Select All	1-6	Interoperation with:
Issues with Miri	Table 8	Select All	7	Did the participant encounter any of these issues with Miri?
			1-6	Issues with Miri
Shifting Ground	N/A	Yes / No	3-6	Shifting ground - Did they express uncertainty about Rust’s semantics or describe how they are either changing or unclear?
			2	Did they express uncertainty about Rust’s semantics?
			1	Uncertainty about Rust’s semantics
Stigma	N/A	Yes / No	2-7	Did they speak about a stigma against unsafe or express shame or negative feelings about unsafe code?
			1	Stigma Against Unsafe
Tools	Table 10	Select All	1-6	Tools
Types	Table 11	Select All	1-6	Types
Unsafe API	N/A	Yes / No	2-7	Did they expose an unsafe API?
			1	Exposed an unsafe API?
Unsafe Data Structure	N/A	Yes / No	2-6	Did they describe an unsafe data structure?
			1	Unsafe data structure?
Unsafe Operations	Table 12	Select All	1	Low-level operations
			2-7	Did they use any of these unsafe operations?
Why unsafe?	Table 13	Select All	1-7	Why unsafe?

Table 3: Codes for the “Bug Finding” theme, which indicate challenges that participants encountered with bug-finding tools such as false positives or false negatives.

Bug Finding	
Rounds	Code(s)
1-6	False Positive, N/A, X missed bug

Table 4: Codes for the “Bugs” Theme, which indicate different types of bugs that were mentioned by interview participants. Cells spanning multiple rows in the “Rounds” column indicate that codes were either merged, split, or replaced in later rounds.

Bugs	
Rounds	Code(s)
1-5	Mismatched Bindings
6-7	Mismatched FFI or Intrinsic Declaration
1-7	Data Races, Deallocate Foreign Memory in Rust, Double Free, Improper Alignment, Incorrect Linking, Incorrect Type Casting, Memory Leak, Undefined Behavior, Unwinding Across FFI Boundaries, Use After Free, Using Uninitialized Memory, Zero-Sized Allocation, From-Raw Wrong Pointer
4-7	Excessive Resource Use, Out of Bounds Access
6-7	Logical Error, Other, Stacked/Tree Borrows Violation

Table 5: Codes for the “Domain” Theme, which indicates the different types of application domains mentioned by interview participants. Cells spanning multiple rows in the “Rounds” column indicate that codes were replaced or split in later rounds.

Domain	
Rounds	Code(s)
1	UEFI/Bootloader
1-7	Embedded Systems
1-3	Web development
4-7	Web Application Development, Web Browser Development
1-7	Databases, JIT Compilation, N/A, Operating Systems, Windows/WinAPI
4-7	Game Development
6-7	Other

Table 6: Codes for the “Generation vs. Validation” Theme, which indicate interview participants’ preferences for using certain type of methods to create declarations for foreign functions.

Generation vs. Validation	
Rounds	Code(s)
1-5	N/A
1-6	Edit the output of a binding generation tool, Use the results from a binding generation tool without modification, Write bindings manually

Table 7: Codes for the “Interoperation” Theme, which indicate the languages that participants used alongside Rust in multi-language applications.

Interoperation	
Rounds	Code(s)
1-6	"WASM / JavaScript / TypeScript", C, C++, Java, Python

Table 8: Codes for the “Issues with Miri” theme, which indicate the difficulties that participants had when validating their test cases using Miri. Cells spanning multiple rows in the “Rounds” column indicate that codes were either merged, split, or replaced in later rounds.

Issues with Miri	
Rounds	Code(s)
1-3	Lack of support for foreign function calls
1-3	Lack of support for inline assembly
4-7	Lack of support for a feature or construct
1-7	Tree Borrows/Stacked Borrows incorrectly considers my unsafe to be in violation of Rust’s aliasing model.
1-7	Runtime overhead

Table 9: Codes for the “Low-Level Operations” theme, which indicate a subset of operations that Rust developers may use in embedded or OS development contexts. After the first round, this theme was merged with the theme “Unsafe Operations”, which is described in Table 12

Low-Level Operations	
Rounds	Code(s)
1	Custom allocator, Inline assembly, Intrinsic, Pointer arithmetic / integer to pointer conversion, System calls

Table 10: Codes for the “Tools” theme, which indicate the different development tools mentioned by interview participants.

Tools	
Rounds	Code(s)
1-6	ASAN, CXX, Clippy, Crucible, DHAT, Diplomat, Emscripten, Kani, Loom, MSAN, Miri, Prusti, PyO3, Shuttle, TSAN, UBSAN, UniFFI, Valgrind, bindgen, cargo fuzz, cbindgen, libfuzzer, wasm-bindgen
2-6	JNA
6	Other

Table 11: Codes for the “Types” theme, which indicate the different Rust data types mentioned by interview participants. The code `ManuallyDrop<T>` was unintentionally left out of round 6.

Types	
Rounds	Code(s)
1-5	<code>ManuallyDrop<T></code>
1-6	<code>Box<T></code> , <code>Cell<T></code> , <code>ExactLen</code> , <code>MaybeUninit<T></code> , <code>Mutex</code> , <code>Non-Null<T></code> , <code>OnceCell<T></code> , <code>Pin<T></code> , <code>RWLock<T></code> , <code>Rc/Arc<T></code> , <code>RefCell<T></code> , <code>Send</code> , <code>Sync</code> , <code>TrustedLen</code> , <code>Vec<T></code> , <code>Weak Reference (Weak<T>)</code>
3-6	Static Variables
6	<code>Layout</code> , <code>LazyCell<T></code> , <code>Other</code> , <code>UnsafeCell<T></code>

Table 12: Codes for the “Unsafe Operations” theme, which indicate the different types of `unsafe` operations mentioned by interview participants. Cells spanning multiple rows in the “Rounds” column indicate that codes were either merged, split, or replaced in later rounds.

Unsafe Operations	
Rounds	Code(s)
1-5	Pointer arithmetic / integer to pointer conversion
1-2	Custom allocator
3-7	Replaced Rust’s default allocator with a custom allocator
1-7	Inline assembly, Intrinsics, System calls
3-7	Raw allocation in Rust
4-7	Transmute
6-7	Multiple / cyclic aliasing, Other

Table 13: Codes for the “Why `unsafe`?” theme, which indicate the different motivations that participants cited for their use of `unsafe` code. Cells spanning multiple rows in the “Rounds” column indicate that a code was replaced in later rounds.

Why unsafe?	
Rounds	Code(s)
1-2	To circumvent the constraints imposed by the API of a dependency or 3rd party
1-5	To circumvent the constraints imposed by Rust’s type system
1-5	To interact with hardware the operating system or other low-level external interfaces
1-5	To interoperate with foreign code
1-7	For fun
1-7	To increase performance
3-7	Unsafe as documentation
1-2	Unsafe is more ergonomic
3-5	Unsafe is more ergonomic or easier to use
6-7	More ergonomic or easier to use
6-7	Impossible or no other choice
6-7	Other (short answer)